

ClearSight Multimodal Network Security

Citation-Grounded, Hallucination-Reduced IoT/OT Threat Detection

Problem framing — July 2026

Anish Bhaskaran, Poongovan Ponnaivaikko, Gangadhar Singh Shiva

Extreme Networks

{abhaskar, pponnava, gshiva}@extremenetworks.com

The Business Problem

Two problems are stacked on top of each other, and neither is fully solved today.

First: IoT devices — smart cameras, sensors, industrial controllers, routers, switches, access points — are everywhere and growing fast, but most organizations barely monitor them the way they monitor laptops or servers. The tools used to catch attacks on this traffic work by counting things: packets per second, byte counts, protocol flags. Someone has to hand-tune what “normal” looks like for every attack type — expensive to build, and it breaks easily the moment traffic looks even a little different from what it was tuned on. In our own testing on a real dataset (CIC-IoT-2023), these tools caught 97.5% of “flood” attacks — the loud, obvious kind, like a garden hose blasting traffic at a target. But for quieter attacks — someone probing a login page, guessing passwords, or forging network addresses — they caught only 2.5–17.5%. The traffic doesn't spike, so these counting-based tools miss it almost entirely, and it's exactly this quieter category that a defender is most likely to miss.

Second: the industry's new fix for this — AI models that can look at a picture of traffic and explain what they see — brings its own risk. These models can look at a rendered traffic picture and give both an answer and a plain-English reason for it, with no hand-built detection rules needed at all. But security teams are using these AI answers faster than anyone can check if they're right. A May 2026 survey of 856 security professionals (ISC2) found widespread worry about AI making things up, and a clear ask for more proof behind AI decisions. A separate 2026 survey (SANS) found 78% of security teams now actively using generative AI day to day, while 63% say it still falls short in real detection and response work. In short: the same AI capability that closes the detection gap opens a new trust gap — and no vendor in this space currently publishes a measured number for how often their AI's explanations are actually right.

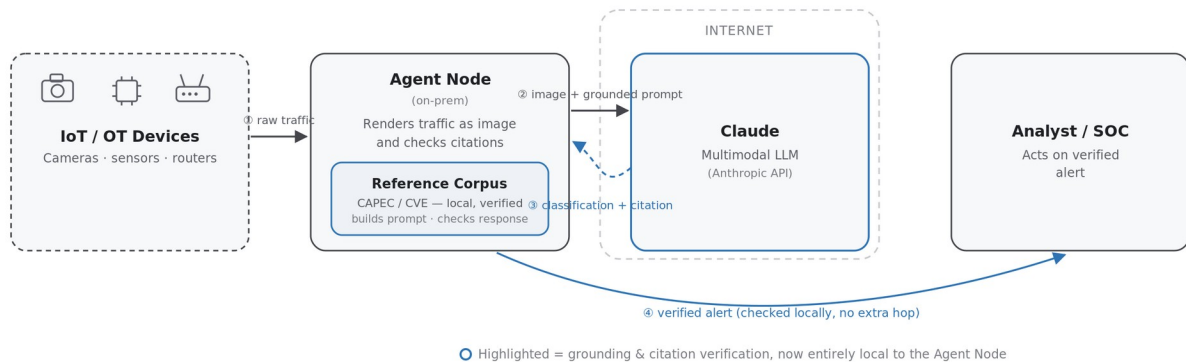
What We're Solving

Our approach uses AI models that can read images and text together. We turn each slice of network traffic into a picture and have the AI classify it directly, with no hand-built detection rules needed at all. The same AI answer also comes with a plain-English reason, pointing to a specific known attack pattern (CAPEC) or vulnerability (CVE). Before that reason ever reaches a security analyst, we check it — in real time — against the official CAPEC and NVD sources to make sure it's real and correctly matched.

Here's the key thing we actually measured, using Claude Sonnet 5 (Anthropic) as the AI model: giving the model a trusted set of reference material to draw from doesn't just make it more accurate — it also teaches it to say “I don't know” instead of guessing when the evidence doesn't support an answer. That one behavior is what turns an AI's answer from a good guess into something an analyst can actually trust and act on.

System Architecture

The diagram below shows how this works end to end. An on-prem Agent Node sits next to the IoT/OT devices, renders their traffic as an image, and holds the hand-verified CAPEC/CVE reference corpus locally. It sends the image plus a grounded prompt to Claude over the internet; Claude returns a classification and a citation. The Agent Node checks that citation against its own local corpus — no extra network hop required — and only then forwards a verified alert to the analyst or SOC.



IoT/OT devices → Agent Node (traffic image + local reference corpus) → Claude over the internet → verified alert to the analyst, checked locally.

Design Trade-off: Full Corpus vs. Retrieval (RAG)

It's worth being precise about what our current design actually does, because the term “RAG” gets used loosely. Right now, every single classification call sends the entire reference corpus — every CAPEC and CVE entry we've hand-verified — into the prompt, in full, along with the traffic image. Claude never sees a partial list; the whole answer key is in front of it every time. That's closer to what's sometimes called “context-stuffing” than to real retrieval-augmented generation (RAG), where a system would first search for just the handful of entries relevant to that specific sample, and send only those.

We chose full-corpus context-stuffing deliberately for this stage of the work, not by accident:

- It guarantees the right answer is always available. If the correct CAPEC or CVE entry exists in our corpus, it is always in front of the model — there's no separate search step that could miss it or rank it too low to include.
- It removes a whole category of failure from this research. We're specifically measuring whether the model can be trusted to cite correctly and decline appropriately when it doesn't have grounds — mixing in retrieval quality as a second variable would make it harder to isolate that.
- It was simple to build and verify. No embeddings, no vector index, no separate retrieval pipeline to test and validate — just one prompt, one API call, one thing to get right.

The cost of that choice: every call resends the full corpus as text, so the prompt's input-token cost and latency scale with corpus size, not with how much of it is actually relevant to that one sample. Today's corpus is small enough that this doesn't matter. It won't stay small forever — every new CAPEC or CVE entry we verify and add makes every future call to Claude larger and more expensive, even for samples that only need one or two of those entries.

A production version would likely move to real RAG: embed the corpus once into a vector index (inside the Agent Node, so it stays local), and at classification time retrieve only the top few entries relevant to that traffic sample instead of sending all of them. The table below summarizes the trade-off:

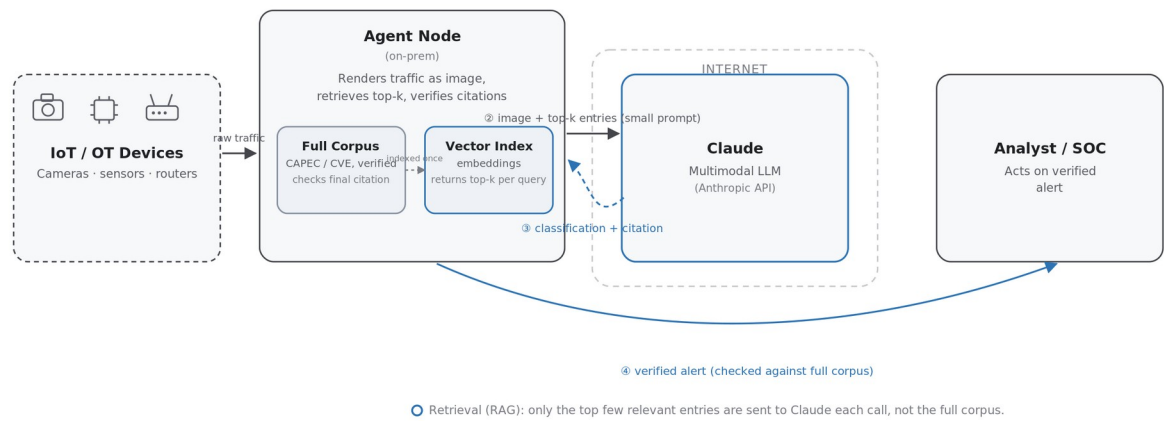
	Full corpus (current)	Retrieval / RAG
What's sent to Claude	The entire verified corpus, every call	Only the top few entries retrieved as relevant to that sample
Main risk	None from missed retrieval — the right entry is always present	The right entry can be missed if retrieval ranks it too low or the search misses it

	Full corpus (current)	Retrieval / RAG
Main cost	Grows with total corpus size, on every single call	Stays small and roughly flat as the corpus grows, plus a one-time indexing cost
Best fit	Small corpus, research-stage measurement of citation trust	Larger, growing corpus, production-scale deployment

For the current stage — proving the citation-verification idea works at all — full-corpus context-stuffing is the right, simpler choice. Retrieval becomes the right choice once the corpus is too large to reliably fit in every prompt, which is a “when we scale up” problem, not a “this doesn't work” problem.

What a RAG-Based Design Would Look Like

If we move to real retrieval, the Agent Node gains two local pieces instead of one: the full verified corpus stays in place (it's still what we check the final citation against), and a vector index gets built from it — a one-time, offline embedding step. At classification time, the Agent Node embeds a query and pulls back only the top few relevant entries from that index, entirely locally, with no network round-trip. Only that smaller set travels to Claude, instead of the whole corpus.



RAG design: Agent Node retrieves the top-k relevant entries locally, then sends only those (plus the traffic image) to Claude — the full corpus is still used to verify the returned citation.

Dataset & Reference Corpus

Every claim in this document rests on two separate sources of data: one for the traffic itself, and one for checking the AI's explanations.

- The traffic data — CIC-IoT-2023. A public dataset of real IoT-botnet traffic, already labeled with 33 attack types across 7 categories (including Mirai and various flood attacks), recorded from a real setup of 105 IoT devices. We use every one of the 34 labels provided (the 33 attack types plus normal/benign traffic) and turn each labeled slice of traffic into the picture the AI classifies. This is real captured traffic — not something we made up or simulated.
- The answer key — hand-checked CAPEC and CVE entries. A separate list we built ourselves, of known attack patterns (CAPEC) and known vulnerabilities (CVE), where we personally checked every entry against the official MITRE and NVD websites before adding it. This same list does two jobs: it's the only thing the AI is allowed to cite from when we want a checked answer ("cite only from this list, or say you don't know"), and it's

also exactly what we compare the AI's answer against afterward to grade it. Because it's the same list both times, the rules and the grading can never quietly drift apart.

- Why we need both. The traffic data alone would only tell us if the AI's classification is right or wrong. The answer-key list alone would just be a reference sheet sitting unused. Put together — real traffic going in, a checked list controlling what can be cited coming out — we can actually measure whether the AI's explanation is grounded in fact, not just whether it sounds convincing.

Why Not a Traditional, Custom-Trained ML Pipeline

It's worth explaining plainly why we didn't start by training our own image-recognition model (like a CNN) from scratch, even though we're already turning traffic into images that such a model could technically use.

- It needs a lot of labeled examples before it works at all. A custom-trained image model needs a big pile of labeled examples for every single attack type before it can recognize even one of them. Whenever a new attack shows up, someone has to collect new examples, label them, and retrain the whole thing before it can spot that attack. The AI approach we tested didn't need any of that — it correctly classified every attack type we tried, including the quiet ones older tools miss, with zero training.
- It needs expensive hardware before you even know if it works. Training and running a custom image-recognition model needs dedicated GPU computers in place first — a real cost paid before there's any proof the idea works. The AI approach we tested needed none of that: it just calls an existing AI service over the internet, so we could test and improve the idea before spending anything on hardware.
- It never stops needing retraining. Every time a new attack shows up or a new type of device joins the network, a traditional model has to be retrained and rechecked — slow, expensive work in a world where new IoT-botnet attacks keep appearing. The AI approach we tested doesn't need any retraining cycle at all.
- That's why we started with an off-the-shelf AI model instead of building our own: it's the fastest, cheapest way to prove the idea works, with no hardware or data-collection costs upfront.

Why Images, Not Raw Packet Captures

A fair question: why turn traffic into a picture at all, instead of just handing the AI the raw captured data? A few clear reasons:

- The input size stays small and predictable. A busy network can generate gigabytes of raw captured data, but a picture of that same traffic is always a small, fixed size no matter how much traffic there was. That keeps speed and cost predictable no matter how loaded the network is — raw data doesn't give you that.
- Patterns are easy to see instead of hidden. Things like repeating flood attacks, shifting traffic types, or changing packet sizes are nearly invisible in raw data, but jump out as obvious shapes once drawn as a picture. That's useful two ways: a human can eyeball the picture and sanity-check it, and an AI that can "see" images can reason over it directly.
- No custom code needed for every protocol. Analyzing raw traffic the old way means building a separate piece of code for every protocol and attack type — the same expensive, fragile pattern described in the Business Problem section above. A picture works the same way no matter what protocol it came from; the AI reasons over the image itself, not some custom parser's output.
- Less sensitive data gets shared. The picture only shows overall traffic patterns (speeds, flags, packet sizes) — not the actual content inside the packets. That means we can send it to a classification service, even an outside API, without exposing sensitive payload data, which matters for privacy and for industrial (OT) environments with strict data rules.

- It works with different kinds of AI models. The same picture can be fed to a general-purpose AI model, as in our current research, or to a specially trained image classifier later — without building a separate data pipeline for each one.

A Real Example: Raw Captured Data vs. the Picture

This isn't hypothetical — our sample manifest records exactly which rows of raw data became which picture. Example 1 below (DDoS-TCP_Flood_0000) is built from rows 102,981–103,011 of the underlying capture file. Here are 3 of those actual rows, each one a single network flow with dozens of numeric columns:

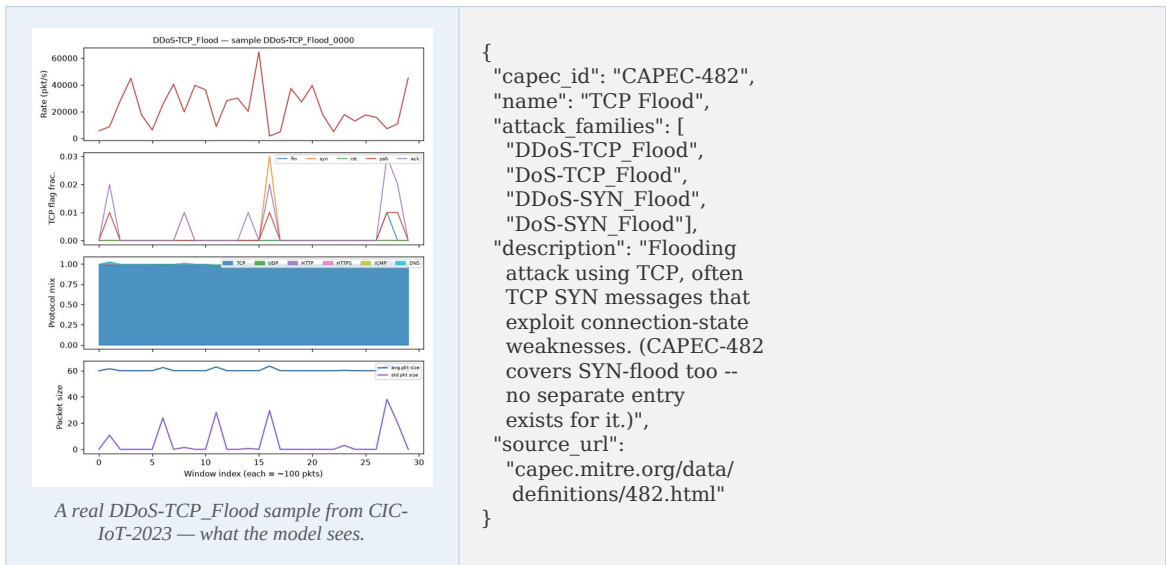
Header Len	Protocol	TTL	Rate (pkts/s)	SYN flag	ACK flag	Tot size
20.0	6 (TCP)	64.0	5585.33	0.0	0.0	60.0
20.12	6 (TCP)	67.44	8667.89	0.0	0.02	61.44
20.0	6 (TCP)	64.0	28151.58	0.0	0.0	60.0

These 7 columns are a small slice — the real file has 39 numeric columns per row, and this one flood-attack capture alone runs to 265,777 rows: about 48 MB of raw data, for a single sample class. The rendered picture the AI actually looks at for the windowed sample built from these rows — Example 1 below — is a fixed 92 KB image, regardless of how many rows the underlying capture contains. That's the size and readability gap in concrete numbers, not just in principle.

Three Concrete Examples

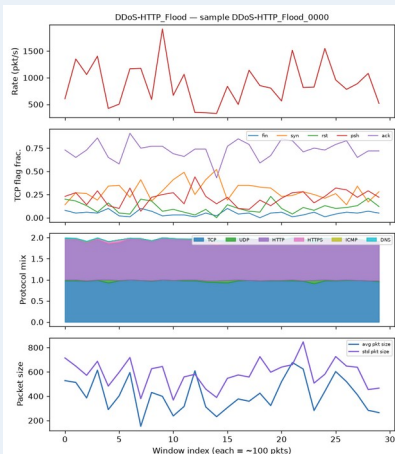
To make this concrete: below are three real examples from our testing. Each one shows the picture the AI actually looked at, next to the real answer-key entry it was allowed to cite for that type of traffic.

Example 1 — TCP flood traffic



This one's a correct match — the picture shows a TCP flood, and CAPEC-482 is exactly the right entry for it. If the AI cites this ID for this picture, we mark it correct.

Example 2 — HTTP flood traffic

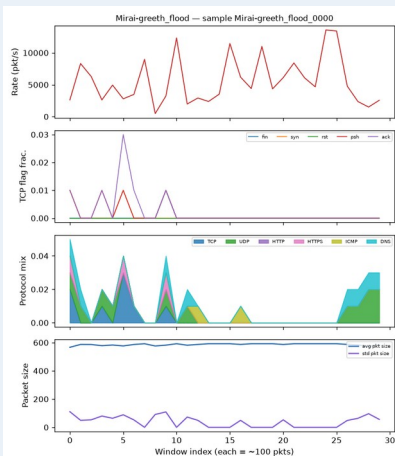


A real DDoS-HTTP_Flood sample — application-layer flooding, visually distinct from the TCP flood above.

```
{
  "capec_id": "CAPEC-488",
  "name": "HTTP Flood",
  "attack_families": [
    "DDoS-HTTP_Flood",
    "DoS-HTTP_Flood"
  ],
  "description": "Flooding
attack using HTTP,
consuming resources at
the application layer
via legitimate session-
based HTTP GET requests
-- which makes it
difficult to detect.",
  "source_url":
    "capec.mitre.org/data/
definitions/488.html"
}
```

Same checking process, but a different type of attack and a different entry, separately verified. The AI has to pick the right one out of the whole list every time — it's never just handed the answer.

Example 3 — Mirai botnet traffic (a CVE, not a CAPEC, example)



A real Mirai-greeth_flood sample — traffic from a device already compromised by the Mirai botnet.

```
{
  "cve_id": "CVE-2016-10401",
  "attack_families": [
    "Mirai-greeth_flood",
    "Mirai-greip_flood",
    "Mirai-udpplain"
  ],
  "description": "ZyXEL
PK5001Z devices have
zyad5001 as the su
password -- easier root
access for remote
attackers. (Relevant to
Mirai's device-compromise
phase, NOT the flood
technique itself.)",
  "source_url":
    "nvd.nist.gov/vuln/
detail/CVE-2016-10401"
}
```

That note in parentheses is there on purpose: CVE-2016-10401 explains how a device gets hijacked into the Mirai botnet — not how the flood attack itself works. Being that precise is what lets our grading catch a sneaky mistake, described below: if the AI used this CVE to explain the flood attack itself instead of the hijacking step, we'd mark it wrong, even though the ID itself is real.

Why It Matters

- The real problem is sneakier than “the AI just made something up.” In our testing with Claude Sonnet 5, the AI rarely invented fake IDs. The bigger problem was picking a real ID and attaching it to the wrong explanation — this happened in 25–50% of answers when the AI wasn't given a checked reference list. That's more dangerous than an obvious fake, because it looks correct at first glance, and it's exactly the kind of mistake a busy analyst would miss.

- You can't fix this by just trusting a good AI model. Even a top-tier model like Claude Sonnet 5, tested carefully across all 34 attack types and three different setups, still gave wrong-explanation citations when it wasn't given a checked reference list. The fix is giving it a checked list and independently verifying its citations — not just trusting a well-known AI brand. That's why we believe a separate verification layer is needed, tested and proven on its own evidence.
- The AI classification itself already works well — trust is the real open question, not capability. Classifying traffic straight from the picture held 96.7–100% accuracy across all 34 attack types we tested, including the ones older tools struggle with most. The real barrier to using this in production isn't whether the AI can correctly classify IoT threats — it's whether we can trust its explanations without a person having to double-check everything by hand.
- This is what the market is already asking for. AI-focused security is the fastest-growing area of cybersecurity investment — \$4.1B in the first quarter of 2026 alone, up 47% from a year earlier — and the surveys above show buyers specifically want AI they can verify, not another black box.
- It's an opening into a big, growing market. Spending on IoT security software is estimated at \$28.7B to \$58.3B (2025/2026, depending how it's measured), growing 18–32% a year, and none of the leading vendors (Claroty, Armis, Nozomi, Dragos) currently publish a measured number for how often their own AI features get things wrong.

What Success Looks Like

Near term (0–12 months) — the research holds up at scale, and becomes a usable tool:

- We repeat the testing at a larger scale on Claude Sonnet 5 and get the same results — almost no invented IDs, and the same measurable improvement in the AI knowing when to say “I don't know.” Then we test on more AI models to confirm it isn't a one-model fluke.
- A working prototype of the verification tool exists, built on the classification-and-citation system and answer-key list we already have.
- The method gets published (peer review or arXiv), so it's backed by outside credibility, not just our own word.

Medium term (12–24 months) — the technology is in someone else's product:

- At least one security vendor integrates our tool into their product, adding verified sourcing to the AI alerts they already generate.
- Real security teams, or managed security providers, are piloting it directly, on exactly the quiet attack types older tools struggle with.
- In real-world use, the invented-ID rate stays at zero and the “correctly says I don't know” rate stays close to what we measured in testing (~40%) — meaning the lab results hold up on real traffic, not just our curated test data.

Long term — the outcome that actually matters:

- Analysts using the tool spend noticeably less time double-checking AI explanations, and fewer decisions get made based on a wrongly-matched citation.
- Checking AI citations becomes a normal, expected part of security tools — not something we have to keep explaining as special, but a standard layer other vendors build on top of.

References

Every factual and statistical claim in this document traces to one of the sources below, consistent with the citation-verification standard the product itself is built around.

Multimodal LLM used for classification & citation testing: Anthropic. Claude Sonnet 5 [large language model], used as the multimodal (vision + text) model for all classification, explanation, and citation-verification testing referenced in this document. — anthropic.com/claude

IoT-botnet traffic dataset: Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A.A. (2023). CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors*, 23(13), 5941. — doi.org/10.3390/s23135941

Attack-pattern reference source: MITRE CAPEC (Common Attack Pattern Enumeration and Classification) — primary source every cited CAPEC entry in this document was individually verified against. — capec.mitre.org

Vulnerability reference source: NVD (National Vulnerability Database) — primary source every cited CVE entry in this document was individually verified against. — nvd.nist.gov

AI hallucination / trust survey (SOC): ISC2 Research, "Rethinking AI's Impact on Cybersecurity Roles," May 2026 (n=856 security professionals). — isc2.org/Insights/2026/07/rethinking-ai-impact-on-cybersecurity-roles

2026 SANS AI Survey: SANS Institute, 2026 AI Survey (78% active GenAI use; 63% report significant AI-driven detection/response shortcomings), summarized via Kiteworks. — kiteworks.com/cybersecurity-risk-management/2026-sans-ai-survey-human-analyst

IoT security market size: MarketsandMarkets, "IoT Security Market Report 2025-2031." — marketsandmarkets.com/Market-Reports/iot-security-market-67064836.html

IoT security market growth (alt.): Fortune Business Insights, IoT Security Market. — fortunebusinessinsights.com/iot-internet-of-things-security-market-103852

CPS / OT protection platform leaders: Gartner Magic Quadrant for CPS Protection Platforms, 2025 (Claroty, Armis, Nozomi Networks, Dragos, Microsoft named Leaders). — gartner.com/reviews/market/cps-protection-platforms

AI-native security VC funding: "AI Cybersecurity Funding Boom 2026" (\$4.1B Q1 2026, 47% YoY growth). — buildmvpfast.com/blog/ai-cybersecurity-funding-boom-exaforce-market-map-2026